

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME:** Operating Systems **COURSE CODE:** CSA0404

---

### COURSE OUTCOME COVERED

*CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

*Assessment Tool Weightage: 30%*

---

**QUESTIONS: 5**

**Total Marks:25**

Annexure A

### INDUSTRY PROBLEM SOLVING TASK

---

#### ***Code of Conduct:***

*I K.Rishika (192525114) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

*K.Rishika*

***Signature of the student:***

## Annexure A

### INDUSTRY PROBLEM SOLVING TASK

| S.No | Industry Problem Solving Task Questions                                                                                                                                                                                                                                     | Marks |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 1    | A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether <b>contiguous, linked, or indexed allocation</b> would be most suitable for improving access speed and justify your choice with reasons.                           | 5     |
| 2    | A data center experiences high disk head movement while processing user requests. Suggest the most appropriate <b>disk scheduling algorithm (FCFS, SSTF, SCAN)</b> to reduce seektime and explain how it improves performance in this scenario.                             | 5     |
| 3    | An operating system uses a bitmap for free space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of <b>bitmap-based allocation</b> and justify your solution.                       | 5     |
| 4    | A real-time application requires consistent and predictable disk access times. Evaluate whether <b>SCAN or C-SCAN</b> would be more suitable and explain your reasoning based on system requirements.                                                                       | 5     |
| 5    | A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors. | 5     |

### RUBRICS FOR CALCULATION

| Criteria                            | Weightage | Level 4 (Exemplary)                                                          | Level 3 (Proficient)                                 | Level 2 (Developing)                            | Level 1 (Beginning)                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding ; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                 | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility         | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools            | Minimal or incorrect use of tools                 |
| Analysis                            | 10%       | Thorough analysis with validated results and performance evaluation          | Good analysis with reasonable validation             | Basic analysis with limited validation          | Poor or no analysis                               |

## 1. Cloud Storage Company – Selecting a File Allocation Method

### Problem Statement

A cloud storage company is facing performance issues because users frequently access files. The company needs to select the most suitable file allocation method among **Contiguous, Linked, and Indexed allocation**.

The major requirements are:

- Fast file access
- Reduced access delay
- Efficient storage utilization
- Good sequential and random access
- Support for frequently accessed files
- Reduced disk-head movement

### Contiguous Allocation

In contiguous allocation, all blocks of a file are stored next to each other.

For example:

File A → 10 → 11 → 12 → 13 → 14

### Advantages

- Very fast sequential access
- Very fast random access
- Simple to implement
- Less disk-head movement
- Excellent locality
- High performance for frequently accessed files

### Disadvantages

- External fragmentation may occur
- File growth can be difficult
- A large continuous free area may not always be available
- Files may need to be relocated

### Linked Allocation

In linked allocation, the blocks of a file can be located anywhere on the disk. Each block contains a pointer to the next block.

Example:

10 → 25 → 7 → 42 → 18

### Advantages

- Files can grow easily

- External fragmentation is reduced
- Blocks do not need to be consecutive
- Efficient use of available free space

#### **Disadvantages**

- Random access is slow
- Pointer storage creates overhead
- Blocks may be located far apart
- More disk-head movement may occur

#### **Indexed Allocation**

In indexed allocation, an index block stores the addresses of all blocks belonging to a file.

#### **Advantages**

- Supports direct/random access
- File blocks do not need to be contiguous
- Files can grow more easily
- Less external fragmentation

#### **Disadvantages**

- Requires additional index blocks
- Extra storage overhead
- Large files may require multiple index structures

#### **Comparison**

| <b>Feature</b>    | <b>Contiguous</b> | <b>Linked</b> | <b>Indexed</b> |
|-------------------|-------------------|---------------|----------------|
| Sequential Access | Excellent         | Good          | Good           |
| Random Access     | Excellent         | Poor          | Excellent      |
| Access Speed      | Very High         | Low           | High           |
| Fragmentation     | High possibility  | Low           | Low            |
| File Growth       | Difficult         | Easy          | Easy           |
| Pointer Overhead  | Low               | High          | Moderate       |

#### **Recommended Solution**

For a cloud storage company where **frequent file access and high speed** are the main requirements, **Contiguous Allocation** is the most suitable choice.

The main reason is that consecutive blocks provide excellent locality. When a file is accessed, the disk does not need to move significantly between blocks.

For example:

File:

100 101 102 103 104

The blocks are close together, so access can be performed quickly.

### **Practical Improvement**

A modern storage system can use **extent-based allocation** instead of requiring the entire file to be stored in one continuous area.

For example:

File A

Extent 1 → 100–149

Extent 2 → 300–349

Extent 3 → 500–529

This combines the performance advantages of contiguous allocation with better flexibility.

### **Conclusion**

**Contiguous or extent-based allocation is recommended for frequently accessed cloud files because it provides fast sequential access, fast random access, excellent locality, and reduced disk-head movement.**

## **2. Data Center – Reducing Disk Head Movement**

### **Problem Statement**

A data center is experiencing high disk-head movement while processing user requests. The possible algorithms are:

- FCFS
- SSTF
- SCAN

The main objective is to reduce seek time and improve performance.

### **FCFS**

FCFS stands for **First Come First Serve**.

Requests are processed in the order in which they arrive.

Example:

Initial Head = 50

Requests:

82, 170, 43, 140, 24

The sequence is:

50 → 82 → 170 → 43 → 140 → 24

### **Advantages**

- Very simple

- Easy to implement
- Fair because requests are processed in arrival order
- No starvation

#### **Disadvantages**

- Can result in high seek time
- Does not consider the location of requests
- Large unnecessary head movements may occur

#### **SSTF**

SSTF means **Shortest Seek Time First**.

It selects the disk request closest to the current head position.

For example:

Head = 50

Requests:

82, 170, 43, 140, 24

The closest request is:

43

because:

$$|50 - 43| = 7$$

The scheduler then selects the next closest request.

#### **Advantages**

- Reduces average seek time
- Improves disk performance
- Reduces unnecessary head movement

#### **Disadvantages**

- Starvation can occur
- A request far away may wait for a long time
- Slightly more complex than FCFS

#### **SCAN**

SCAN is also called the **Elevator Algorithm**.

The disk head moves in one direction and services requests along the way. When it reaches the end, it reverses direction.

The head services requests while moving toward one end and then comes back.

#### **Advantages**

- Better fairness
- Lower seek time than FCFS
- Suitable for heavy workloads
- Reduces starvation

#### Disadvantages

- The head may travel to the disk boundary
- Some requests may wait longer
- More complex than FCFS

#### Comparison

| Algorithm | Seek Time    | Fairness    | Starvation | Complexity |
|-----------|--------------|-------------|------------|------------|
| FCFS      | High         | Excellent   | No         | Low        |
| SSTF      | <b>Low</b>   | Moderate    | Possible   | Moderate   |
| SCAN      | Low/Moderate | <b>High</b> | Low        | Moderate   |

#### Recommended Solution

For the given problem, **SSTF is recommended when the main objective is reducing average seek time.**

For example:

Head = 50

Requests:

10, 45, 48, 90, 150

SSTF starts with:

50 → 48 → 45 → 10 → 90 → 150

The scheduler continuously chooses the nearest request.

This reduces unnecessary movement and improves average disk response time.

#### Practical Improvement

To prevent starvation, an advanced system can combine SSTF with **aging**. Requests that have waited for a long time can gradually receive higher priority.

#### Conclusion

**SSTF is the most suitable choice when reducing seek time is the primary objective. SCAN can be preferred when fairness and predictable servicing are more important.**

### 3. Large Data Center – Optimizing Bitmap Allocation

#### Problem Statement

An operating system uses a bitmap for free-space management. When disk capacity becomes very large, allocation becomes slow because the system may need to search a large number of bitmap entries.

#### Bitmap Technique

In bitmap allocation, every disk block is represented by one bit.

0 = Free

1 = Allocated

For example:

Block: 0 1 2 3 4 5 6 7 8 9

Bitmap: 1 0 1 1 0 0 1 1 0 0

Free blocks are:

1, 4, 5, 8, 9

The operating system searches the bitmap to find free blocks.

### **Problem With a Simple Bitmap**

Consider a very large disk:

10 million blocks

If the system needs 100 free blocks, searching the bitmap from the beginning every time can be inefficient.

This can increase:

- Allocation time
- CPU usage
- Disk management overhead
- Response time

### **Recommended Optimization – Hierarchical Bitmap**

A **hierarchical bitmap** can be used to improve allocation speed.

Instead of searching every individual block, disk blocks can be divided into groups.

For example:

Group 1 → Blocks 0–99

Group 2 → Blocks 100–199

Group 3 → Blocks 200–299

Group 4 → Blocks 300–399

A higher-level bitmap indicates which groups contain free blocks.

Example:

Group Bitmap:

1 0 1 1 0 1

Here:

1 = Group contains free space

0 = Group is full



The operating system first searches the group bitmap and then searches only the selected group.

### **Advantages**

#### **Faster Search**

The operating system does not need to scan the complete bitmap.

#### **Lower CPU Overhead**

Only relevant groups are searched.

#### **Better Scalability**

It works efficiently for very large disks.

#### **Faster Allocation**

Free blocks can be located more quickly.

#### **Additional Improvements**

The system can also maintain:

- Number of free blocks
- Free-block ranges
- Recently used free locations
- Group-level summaries
- Cached free-space information

These techniques further reduce allocation overhead.

### **Conclusion**

**Hierarchical bitmap allocation is recommended for large-scale storage systems because it reduces the search area and improves free-space allocation efficiency.**

## **4. Real-Time Application – SCAN vs C-SCAN**

### **Problem Statement**

A real-time application requires:

- Consistent disk access
- Predictable response time
- Fair request servicing
- Reduced variation in waiting time

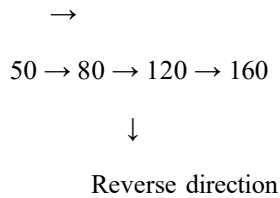
The system must choose between **SCAN** and **C-SCAN**.

### **SCAN**

SCAN moves the disk head in one direction and services requests. When it reaches the end, it reverses direction.

Example:

0 ----- 199



### Advantages

- Good fairness
- Reduced starvation
- Better than FCFS for heavy workloads
- Reasonable waiting-time performance

### Disadvantages

- Requests may have different waiting times
- The head changes direction
- Predictability is lower than C-SCAN for some workloads

### C-SCAN

C-SCAN means **Circular SCAN**.

The head services requests in only one direction. After reaching the end, it returns to the beginning and starts again.

Example sequence:

50 → 80 → 120 → 160 → 190 → 0 → 20 → 40

### SCAN vs C-SCAN

| Feature               | SCAN            | C-SCAN                  |
|-----------------------|-----------------|-------------------------|
| Direction             | Both directions | One servicing direction |
| Fairness              | Good            | Very Good               |
| Predictability        | Good            | <b>Very Good</b>        |
| Waiting Time          | Variable        | More uniform            |
| Starvation            | Low             | Very Low                |
| Real-Time Suitability | Good            | <b>Excellent</b>        |

### Recommended Solution – C-SCAN

For a real-time application, **C-SCAN is more suitable** when consistent and predictable disk access is the main requirement.

#### Reason 1 – Uniform Waiting Time

Requests are handled in a circular manner, reducing differences in waiting time.

#### Reason 2 – Predictable Servicing

The disk head follows a consistent servicing direction.

### **Reason 3 – Better Fairness**

Requests distributed across the disk receive more uniform treatment.

### **Reason 4 – Suitable for Time-Sensitive Systems**

C-SCAN can be useful in systems where consistent response is more important than achieving the absolute minimum average seek distance.

Examples include:

- Real-time monitoring
- Industrial systems
- Transaction processing
- Time-sensitive data collection
- Streaming workloads

### **Conclusion**

**C-SCAN is the preferred choice for a real-time system requiring consistent and predictable disk access times.**

## **5. Hybrid Storage System – File Allocation + Disk Scheduling**

### **Problem Statement**

A company wants to design a hybrid storage system that combines file allocation and disk scheduling techniques for large-scale data access.

The possible choices are:

### **File Allocation**

- Contiguous
- Indexed

### **Disk Scheduling**

- FCFS
- SSTF

The objective is to achieve:

- High access speed
- Low seek time
- High throughput
- Efficient storage utilization
- Scalability

### **Recommended Combination**

#### **Extent-Based Contiguous Allocation + SSTF Scheduling**

This combination provides a good balance between fast file access and reduced disk-head movement.

### Why Contiguous Allocation?

Suppose a file requires five blocks:

200 201 202 203 204

The blocks are adjacent.

Therefore, accessing the file requires very little disk movement.

#### Benefits

- Fast sequential access
- Fast random access
- Excellent locality
- Low seek overhead
- High throughput

However, pure contiguous allocation can suffer from fragmentation.

Therefore, a modern system can use **extent-based allocation**.

Example:

File A

Extent 1 → 200–249

Extent 2 → 400–449

Extent 3 → 700–729

Each extent contains consecutive blocks.

### Why SSTF?

SSTF selects the request closest to the current head.

For example:

Head = 180

Requests:

200, 201, 202, 203, 204

The sequence becomes:

180 → 200 → 201 → 202 → 203 → 204

The total movement is:

$20 + 1 + 1 + 1 + 1 = 24$  blocks

This is efficient because the file blocks are located close together.

### Comparison of Combinations

| Allocation | Scheduling | Access Speed | Seek Time | Fragmentation | Scalability |
|------------|------------|--------------|-----------|---------------|-------------|
|------------|------------|--------------|-----------|---------------|-------------|

|            |      |                  |            |             |             |
|------------|------|------------------|------------|-------------|-------------|
| Contiguous | FCFS | High             | High       | Medium/High | Medium      |
| Contiguous | SSTF | <b>Very High</b> | <b>Low</b> | Medium      | High        |
| Indexed    | FCFS | Medium           | High       | Low         | High        |
| Indexed    | SSTF | High             | Low        | <b>Low</b>  | <b>High</b> |

### Proposed Hybrid Architecture

This architecture combines several operating system techniques.

### Performance Analysis

#### 1. Access Time

Extent-based allocation keeps related blocks close together, reducing access time.

#### 2. Seek Time

SSTF chooses nearby requests, reducing unnecessary head movement.

#### 3. Fragmentation

Extents reduce some of the limitations associated with requiring one huge continuous region.

#### 4. Storage Utilization

A hierarchical bitmap helps identify available areas efficiently.

#### 5. Scalability

The combination is suitable for large-scale systems because each component addresses a different storage-management problem.

### Overall Comparison of the Five Problems

| Industry Problem      | Recommended Technique     | Main Objective             |
|-----------------------|---------------------------|----------------------------|
| Cloud storage         | Contiguous / Extent-Based | Fast file access           |
| Data center           | SSTF                      | Reduce seek time           |
| Large bitmap          | Hierarchical Bitmap       | Fast free-space allocation |
| Real-time application | C-SCAN                    | Predictable access         |
| Hybrid storage        | Extent-Based + SSTF       | High overall performance   |

### Overall Engineering Analysis

The assessment demonstrates that different applications require different storage-management strategies.

#### For maximum file access speed

**Contiguous allocation** is suitable because blocks are stored close together.

#### For minimum average seek time

**SSTF** is effective because it services the closest request first.

### **For large-scale free-space management**

**Hierarchical bitmap** improves the efficiency of locating free blocks.

### **For predictable real-time access**

**C-SCAN** provides more uniform servicing compared with **SCAN**.

### **For large-scale hybrid storage**

**Extent-based allocation + SSTF** provides a strong combination of locality and reduced seek movement.

### **Final Conclusion**

File allocation, free-space management, and disk scheduling are important components of operating system performance. The correct selection of these techniques can significantly improve storage efficiency, reduce access delays, and increase system throughput.

For the cloud storage problem, **contiguous or extent-based allocation** is recommended because it provides fast access and excellent locality. For a data center with high disk-head movement, **SSTF** is suitable for reducing average seek time. For large disks, **hierarchical bitmap management** improves the efficiency of free-space allocation by reducing unnecessary bitmap searches.

For real-time applications, **C-SCAN** is preferred when predictable and uniform disk access is required. Finally, a hybrid system using **extent-based allocation with SSTF scheduling** provides an effective combination for large-scale data access.

Therefore, an efficient modern storage system should select algorithms according to its workload and performance requirements rather than using a single algorithm for every situation.

### **Final Recommended Solutions**

#### 1. Cloud Storage

→ Contiguous / Extent-Based Allocation

#### 2. Data Center

→ SSTF Scheduling

#### 3. Large Bitmap Management

→ Hierarchical Bitmap

#### 4. Real-Time Application

→ C-SCAN

#### 5. Hybrid Storage System

→ Extent-Based Allocation + SSTF